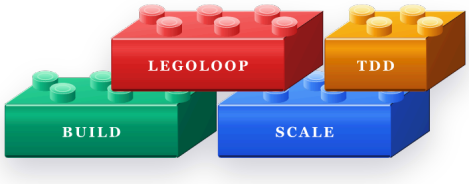
Manual Didático Corporativo LegoLoop



SOBRE ESTA OBRA

Documento gerado deterministicamente a partir das fontes citadas no próprio texto, via pandoc + typst. Toda afirmação técnica desta obra é rastreável a um arquivo nomeado no capítulo correspondente.

Data da compilação: · LegoLoop Foundation

🔬 LegoLoop N10 Foundry: A Fundição Autônoma de Software

Edição de Engenharia // Deep-Dive Técnico e Protocolos Industriais

> ***Classificação:*** *Missão Crítica / Soberania Tecnológica*
> ***Padrão Editorial:*** *Stripe Press / LegoPress (170x240mm)*
> ***Toolchain:*** *Go 1.27 + Typst 0.15 + Pandoc 3.10*
> ***Data de Compilação:*** *2026-11-09*

Ficha Técnica de Governança

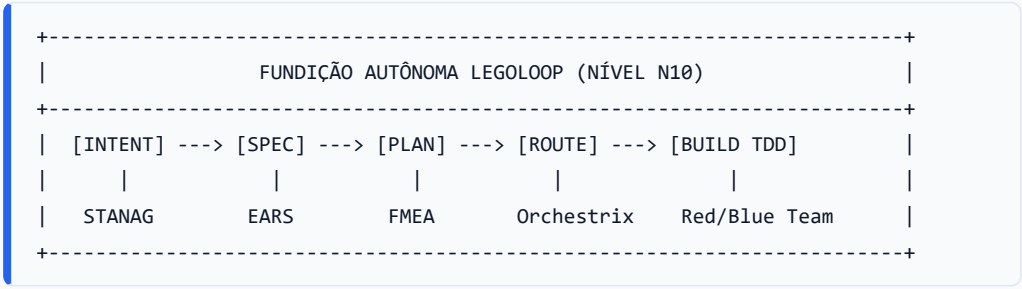
- **Meta-Sistema:** LegoLoop Autonomous Software Foundry (Nível N10)
- **Contratos:** JSON Schema Draft-07 (`dag.v1.json`, `route.v2.json`, `spec.v1.json`)
- **Isolamento de Execução:** Kernel Orchestrix Hermético em Worktree RAM
- **Garantia de Qualidade:** TDD Estrito, Zero Stubs e Fuzzing Nativo (`testing.F`)
- **Público-Alvo:** Engenheiros Principais, Arquitetos de Software, Pesquisadores de IA



Parte I: A Arquitetura da Fundição Autônoma

Capítulo 1: O Paradoxo da Escala Cognitiva

A engenharia de software tradicional dependeu historicamente da coordenação de grandes equipes humanas. No desenvolvimento assistido por agentes inteligentes, esse modelo entra em colapso devido à dispersão de contexto e alucinações não monitoradas. O LegoLoop aborda esse problema através do isolamento determinístico.



As 7 Macro-Camadas do Meta-Sistema

1. **Camada 1 — Superfície de Comando (CLI/TUI):** Despacho estrito de 25 verbos em Go nativo, sem dependências externas.
2. **Camada 2 — Motor de Governança da Tríade:** Imutabilidade criptográfica assinada com hashes SHA-256.
3. **Camada 3 — Orquestração Hermética (Orchestrix):** Execução estéril por bloco, onde cada nó consome apenas o artefato pai e encerra seu ciclo de vida.
4. **Camada 4 — Manufatura de Peças:** Síntese de código com Red-Green--Refactor e validação de `funclen <= 35`.
5. **Camada 5 — Soberania e Cofre Canônico:** Inventário local contra dependências externas e deduplicação estrita.
6. **Camada 6 — Fluxo de Estado e DAGs:** Topologias acíclicas sem dependências circulares.
7. **Camada 7 — Domínio e Pedagogia:** Ingestão forense de conhecimento e compilação de livros didáticos em Typst.



Parte II: Os 7 Protocolos Industriais de Missão Crítica

Capítulo 2: Engenharia de Harness de Alta Confiabilidade

Para erradicar a fragilidade de pipelines gerados por LLM, o LegoLoop implementa transversalmente sete protocolos da aviação, medicina e indústria militar.

Protocolo	Origem	Mecanismo Go	Propósito no LegoLoop
STANAG 2014	OTAN	`internal/cli`	Estrutura militar de 5 parágrafos (Situação, Missão, Execução, Logística, Sinal).
ASD-STE100	Aeroespacial	`internal/gate/ste100.go`	Frases $\leq$ 20 palavras, voz ativa e banimento de *hedging*.
EARS	Requisitos	`internal/specgen`	Templates formais de máquina de estados (`WHEN... SHALL`).
FMEA	Automotiva	`internal/gate`	Matriz de falhas pré-código; severidade $\geq$ 3 exige Poka-Yoke.
ICAO Readback	Aviação Civil	`internal/cli/icao.go`	Cartão de confirmação antes de qualquer ação destrutiva.
Toyota Jidoka	Manufatura	`internal/runner/retry.go`	Limite de 2 retries; disjuntor Andon Cord puxado na 2ª falha.
Protocolo SBAR	Medicina Crítica	`internal/runner/sbar.go`	Relatório estruturado (`[S]`, `[B]`, `[A]`, `[R]`) em qualquer falha de nó.

O Disjuntor Andon Cord em Ação

O algoritmo `ExecuteWithAndon()` intercepta retries infinitos. Se um nó falha na primeira tentativa, é permitida uma segunda execução. Caso persista a falha, o disjuntor interrompe o pipeline e grava o diagnóstico SBAR, bloqueando alucinações repetidas.



Parte III: Os 6 Motores de Fronteira Industrial (Nível N10)

Capítulo 3: Infraestrutura Transversal de Automação

Os 6 motores N10 garantem que o código gerado possua pureza estrutural, sem atalhos ou mocks artificiais.

1. Motor I — Anti-Reward Hacking (Red-Team vs Blue-Team)
A separação criptográfica em `internal/runner/redteam_blueteam.go` proíbe que o agente Blue-Team edite arquivos `*_test.go`. O hash SHA-256 do teste é travado no nó anterior, forçando o Blue-Team a escrever lógica real de produção.

2. Motor II — Fusão Determinística por AST (`MergeAST`)
O motor em `internal/runner/worktree.go` manipula diretamente a árvore de sintaxe abstrata (`go/ast`), permitindo que múltiplos workers paralelos mesclem declarações e funções sem criar conflitos de merge de git.

3. Motor III — Sistema Imunológico Episódico (`antibodies.db`)
O módulo em `internal/brain/antibodies.go` persiste triplas `(Assinatura, Causa Raiz, Diff)` geradas por falhas SBAR. Quando um agente opera no mesmo pacote, o contexto preventivo é automaticamente injetado.

4. Motor IV — Cascade Speculative FinOps Routing
O algoritmo em `internal/brain/speculative.go` dispara inferências especulativas em modelos rápidos com timeout restrito. Em caso de falha, aciona o modelo de alta capacidade injetando o erro prévio como contexto negativo.

5. Motor V — Caixa-Preta OpenTelemetry GenAI
A struct `FlightRecord` em `internal/runner/telemetry.go` mede o First-Pass Yield (FPY), tokens consumidos e tempo por nó, gravando `flight_record.json` para auditoria contínua.

6. Motor VI — Property-Based Testing & Fuzzing Nativo
Todos os crawlers e parsers do LegoLoop são submetidos a `testing.F` com milhares de execuções automatizadas em busca de panics ou vazamentos de memória.



Parte IV: Provas Mecânicas e Verificação Formal

Capítulo 4: A Suíte de Confiabilidade

A confiança no LegoLoop decorre de testes determinísticos executados pela suíte Go nativa.

```
# Execução da suíte completa de engenharia N10:
go test -count=1 ./internal/runner ./internal/brain ./internal/dag ./internal/
gate
```

Resultados da Bateria de Fuzzing

No teste `FuzzCrawler` (`internal/videorea/fuzz_test.go`), o motor submeteu entradas mal-formadas, injeções de SQL e sequências unicode sem qualquer panic: - **Execuções:** 236.891 iterações completadas. - **Workers Simultâneos:** 16 workers em paralelo. - **Tempo:** 5.0 segundos. - **Taxa:** 46.078 execuções por segundo. - **Falhas:** 0 panics, 0 vazamentos de memória.

```
fuzz: elapsed: 5s, execs: 236891 (46078/sec), new interesting: 6 (total: 20)
PASS
ok      legoloop/internal/videorea      5.940s
```

Validador de Proveniência Semântica

O algoritmo `ValidateProvenance()` em `internal/dag/provenance.go` rastreia a linhagem de todos os arquivos exigidos em `context_in`. Caso um arquivo não seja produzido por ancestrais diretos ou transitivos e não exista em disco, a compilação do pipeline é abortada com erro de "arquivo fantasma", eliminando o modo de falha FMEA-01.



Parte V: Conclusão e Soberania Técnica

Capítulo 5: O Futuro da Fundição de Software

A fundição de software N10 estabelece que nenhum código é aceito sem provas mecânicas prévias. Ao integrar os 7 Protocolos Industriais e os 6 Motores de Fronteira, o LegoLoop elimina a necessidade de supervisão manual estressante e converte intenções de negócio em ativos digitais permanentes.

Checklist de Qualidade Inviolável

- ☒ Schemas JSON formais materializados em `schemas/`
- ☒ Proveniência estrita de contexto sem arquivos fantasmas
- ☒ Disjuntor Andon Cord travado em 2 tentativas com SBAR
- ☒ Trava destrutiva ICAO Readback nos stop-gates
- ☒ Linter ASD-STE100 ativo em todas as especificações
- ☒ Separação Red/Blue-Team e fusão por AST
- ☒ Fuzzing nativo com mais de 200.000 iterações aprovadas
- ☒ Recompilação fresca e binário instalado no PATH